Assignment-4
Parallel and Distributed Computing Submitted by:

Kartik Goel
102303182

Q1: Ring Communication

Objective: Write an MPI program where each process sends a message to the next process in a ring topology.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup	Efficiency	Comm %
1	< 0.000001	N/A	N/A	0.00%
2	0.000006	~0.00	~0.00%	100.00%
4	0.000012	~0.00	~0.00%	100.00%

Execution Output:

Process 0 started with value: 100
Process 1 received value: 100
Process 2 received value: 101
Process 3 received value: 103
Process 0 received final wrapped value: 106

Analysis & Key Inferences:

- **Pure Latency Scaling.** The recorded time for P=1 is 0.000000 seconds, reflecting the fact that local arithmetic operations occur in mere fractions of a microsecond. Consequently, the durations measured for P=2 and P=4 are essentially measures of pure MPI communication latency. With the time nearly doubling from P=2 (0.000006s) to P=4 (0.000012s), it is evident that ring topologies necessitate a strict O(p) serialization of sequential communication.
- **Overhead Dominance.** Because the arithmetic payload as of by adding the rank is infinitesimally small, the communication overhead is 100%. The system spends its entire execution time routing MPI_Send and MPI_Recv packets through the middleware.
- **System Design Takeaway:** Ring communication provides excellent ordered

synchronization but zero computational acceleration. It should be used for token-passing or bounds-checking, not for speeding up data processing.

Q2: Parallel Array Sum

Objective: Compute the sum of an array of size 100 in parallel using MPI_Scatter and MPI_Reduce.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup (Sp)	Efficiency (Ep)	Comm %
1	0.000013	1.00	100.00%	0.00%
2	0.000080	0.16	8%	90.59%
4	0.000069	0.18	4.50%	94.67%

Execution Output:

Global Sum: 5050 (Expected: 5050)
Average Value: 50.50

Analysis & Key Inferences:

- **Scalability Limitations:** For a small workload of 100 elements, a single-core sequential summation is completed in a negligible 0.000013 seconds. In this context, the overhead of the MPI stack required to distribute batches of 25 or 50 integers is counterproductive, resulting in a marked degradation of program performance.
- **Variability in OS Scheduling.** A marginal performance gain was observed at P=4 (0.000069s) compared to P=2 (0.000080s). At this microscopic scale, MPI performance is primarily dictated by cache allocation and operating system thread scheduling rather than raw computational power. Despite the slight improvement, communication latency remains the overriding factor, accounting for approximately 94% of the execution time.
- **Strategic Design Principle:** Surpassing a Speedup of 1 requires significant upward scaling of array dimensions (e.g., $N = 10^7$). This expansion is necessary to extend the computational phase sufficiently to offset the inherent communication latencies introduced by MPI_Scatter and MPI_Reduce.

Q3: Finding Maximum and Minimum

Objective: Generate 10 random numbers per process and find the global maximum and minimum values using MPI_MAXLOC and MPI_MINLOC.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup (Sp)	Efficiency (Ep)	Comm %
1	0.000004	1.00	100.00%	0.00%
2	0.000030	0.13	6.66%	92.42%
4	0.000014	0.28	7%	92.19%

Execution Output:

Process 0 generated: 897 562 661 317 604 148 935 477 177 905
Process 1 generated: 215 24 343 559 774 486 720 151 126 890
Process 2 generated: 382 991 223 209 16 206 946 625 650 527
Process 3 generated: 885 201 775 786 493 220 764 136 328 468
--- Global Results —
Global Maximum: 991 (Found in Process 2)
Global Minimum: 16 (Found in Process 2)

Analysis & Key Inferences:

- **Structural Data Overhead.** Employing MPI_2INT for the transmission of specialized val_rank structures to monitor locations results in a marginally larger packet payload than that of a basic scalar reduction. This increased size is a key factor in the substantial communication overhead, which exceeds 92%.
- **Efficiency of Multi-core Caching.** A notable reduction in execution time was observed, dropping from 0.000030 s at P=2 to 0.000014 s at P=4. This improvement stems from the handling of CPU interrupts triggered during random number generation. By distributing these interrupts across four physical cores, the hardware processed the generation phase with sufficient speed to partially mitigate the latencies inherent in MPI reduction.

Q4: Parallel Dot Product

Objective: Compute the dot product of two vectors of size 8 in parallel.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup (Sp)	Efficiency (Ep)	Comm %

1	0.000011	1.00	100.00%	0.00%
2	0.000035	0.31	15.50%	83.75%
4	0.000083	0.13	3.25%	96.26%

Execution Output:

Parallel Dot Product Result: 120
Expected Result: 120

Analysis & Key Inferences:

- **Significant Fine-Grained Imbalance.** With Vector A and Vector B being exceptionally small (N=8), the workload is minimal. In a 4-process configuration, each individual node performs only two multiplications before reaching the MPI_Reduce barrier.
- **Occurrence of Negative Scaling.** The transition from 2 to 4 processes caused the execution duration to rise from 0.000035 s to 0.000083 s, while efficiency plummeted to 3.25%. This regression is attributed to the sequential bottleneck created by the time needed to scatter arrays and collect results.
- **Core System Design Insight:** When the computational requirements are nearly non-existent, distributing a workload across a cluster becomes inherently counterproductive. The resulting network overhead creates a performance bottleneck that actively hinders the system. Consequently, a standalone processor consistently exceeds the performance of an MPI-based setup for microscopic data scales.